



Rapport SAE 5.01

22.11.2024

—

BOUGUERRA Noa

ER Mikail

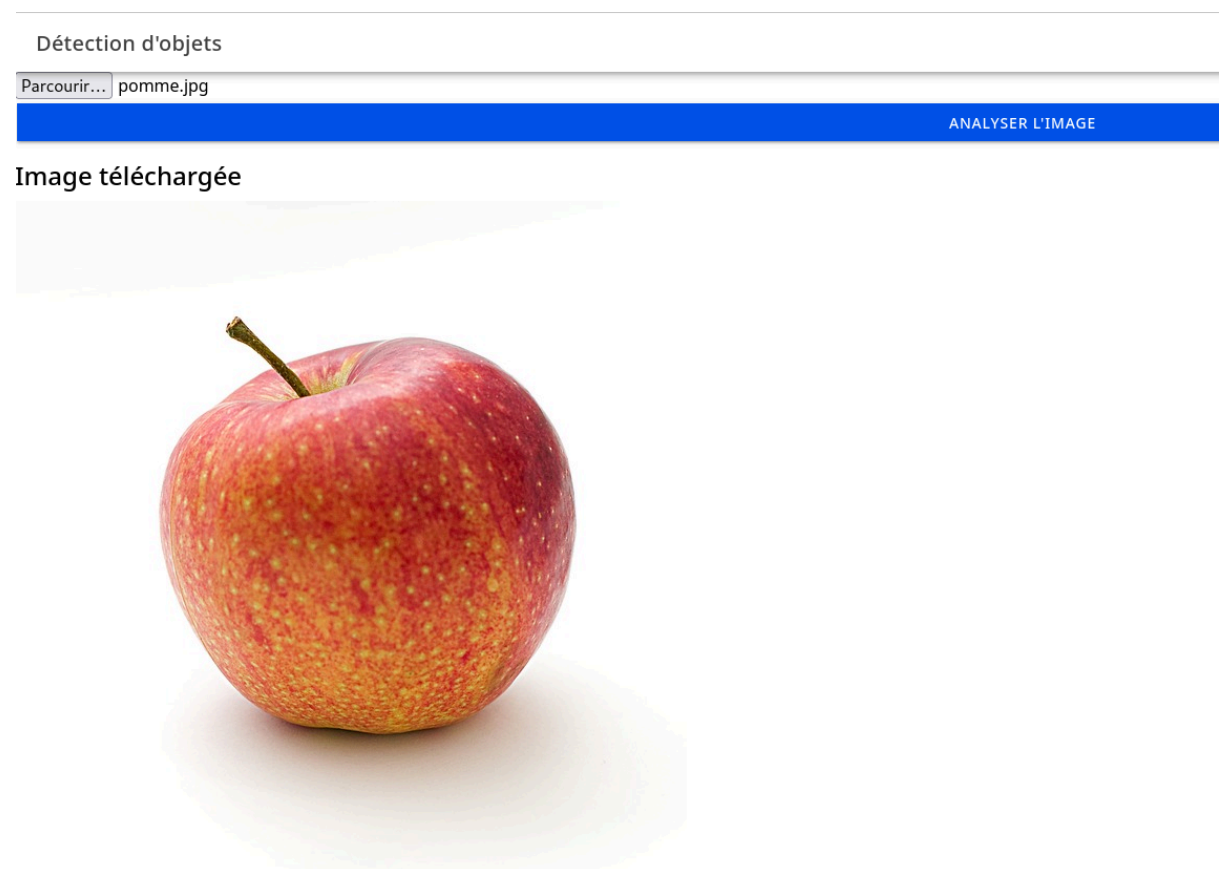
FERCHACH Iliass

KNOFFEL Xavier

Avancement du projet :

Cette semaine, nous avons continué d'essayer d'implémenter YOLO sur Ionic en utilisant TensorFlow.js ou TensorFlow Lite. Malgré nos efforts, nous n'avons pas réussi à l'intégrer de manière fonctionnelle dans l'application.

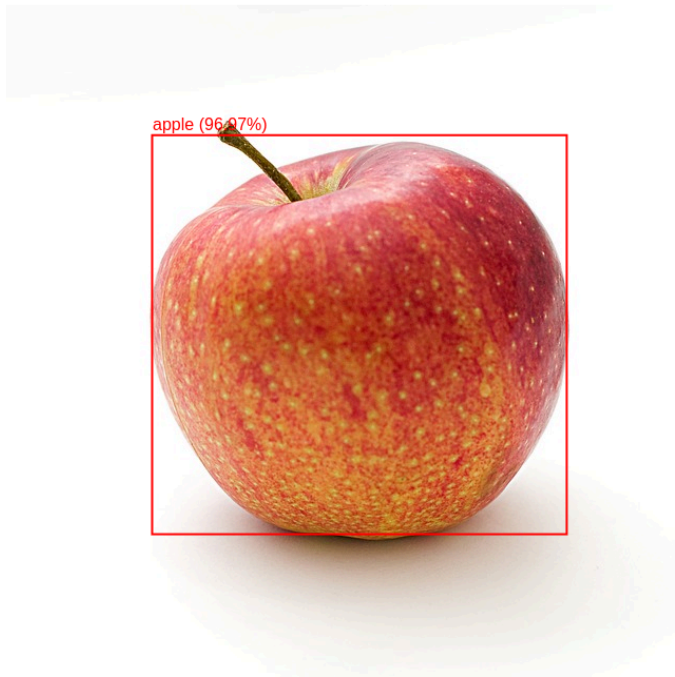
Cependant, nous avons exploré des modèles fournis par TensorFlow, notamment le modèle **Coco SSD**, qui est un modèle pré-entraîné disponible via la bibliothèque TensorFlow.js. L'intégration de ce modèle dans l'application s'est révélée simple et rapide. Les tests effectués ont donné des résultats satisfaisants, comme vous pouvez le voir ci-dessous, avec une détection correcte des objets, et des scores de fiabilité bien interprétés.



Résultats

Classe : apple
Confiance : 96.97 %

Image annotée



Voici un tableau qui compare les modèles d'ia provenant de Tensorlow et YOLO :

Critères	TensorFlow (Coco SSD et modèles similaires)	YOLO
Facilité d'intégration	Très simple à intégrer dans une app mobile avec TensorFlow.js ou TensorFlow Lite. Bien documenté avec des exemples pour Ionic.	Plus complexe, nécessite des conversions. La configuration peut poser problème.
Performance en temps réel	Bonne sur des appareils mobiles récents, mais légèrement moins rapide que YOLO pour des	Excellente, optimisée pour des tâches en temps réel avec un équilibre entre vitesse et précision.

	modèles lourds ou avec de nombreux objets à détecter.	
Précision des résultats	Moins précis que YOLO, surtout sur des objets petits ou très proches. Toutefois, suffisant pour des tâches générales.	Plus précis, particulièrement pour des objets petits ou denses, grâce à son architecture avancée
Compatibilité avec Ionic	Très bonne : TensorFlow.js et TensorFlow Lite s'intègrent nativement avec Angular/Ionic	Moyenne : YOLO doit être converti pour fonctionner avec TensorFlow.js ou TensorFlow Lite, ce qui peut engendrer des erreurs.
Modèles pré-entraînés disponibles	Plusieurs modèles comme Coco SSD, MobileNet et EfficientDet. Coco SSD est optimisé pour 80 classes courantes (personnes, objets, animaux, etc.).	Pré-entraîné pour de nombreuses classes via des poids publics (YOLOv5, v8, etc.). Plus flexible pour les applications spécialisée
Capacité de réentraînement	Possibilité de réentraîner avec TensorFlow sur des datasets personnalisés.	Très flexible. YOLO peut être ré-entraîné sur un dataset personnalisé pour des besoins spécifiques, mais nécessite plus de ressources.
Taille du modèle	Adapté aux appareils mobiles avec des ressources limitées.	Plus volumineux, mais optimisé pour des performances en temps réel. Peut nécessiter plus

		de mémoire et de puissance de calcul.
Prise en charge des frameworks mobiles	Parfaitement intégré dans les bibliothèques TensorFlow Lite et TensorFlow.js, conçues pour les mobiles.	Nécessite des conversions manuelles pour fonctionner avec TensorFlow Lite ou TensorFlow.js sur mobile.
Consommation de ressources (CPU/GPU)	Moins gourmand en ressources, adapté pour des appareils d'entrée ou milieu de gamme.	Plus gourmand, idéal pour les appareils avec GPU dédié ou processeurs performants.
Temps de traitement	Temps de traitement légèrement plus long que YOLO sur des tâches complexes.	Optimisé pour des détections rapides, sur n'importe quel flux.